

YOUTUBE SHORTS

RECOMMENDATION SYSTEM

Complete Production Guide

VOLUME 3

Serving, Deployment & Operations

2025-2026 Edition

Model Serving | A/B Testing | Monitoring | MLOps | CI/CD Pipelines

Table of Contents

Chapter 13: Model Serving Architecture 3

Chapter 14: Real-Time Inference Pipeline 10

Chapter 15: A/B Testing Framework 16

Chapter 16: Monitoring & Observability 22

Chapter 17: MLOps & CI/CD Pipelines 28

Appendix E: Deployment Checklist 34

Appendix F: Incident Response Playbook 36

Chapter 13: Model Serving Architecture

13.1 Serving Infrastructure Overview

Serving recommendation models in production requires a carefully designed infrastructure that balances latency, throughput, availability, and cost. The architecture must handle millions of requests per second while maintaining sub-100ms response times.

The serving stack consists of multiple layers: an API gateway for request routing and rate limiting, a feature store for real-time feature retrieval, model servers for inference, and caching layers to reduce redundant computation.

13.2 Model Server Options

13.2.1 TensorFlow Serving

TensorFlow Serving is the standard choice for TensorFlow models. It provides model versioning, automatic batching, and gRPC/REST APIs out of the box.

TensorFlow Serving Configuration

```
# model_config.pbtxt

model_config_list {

  config {

    name: 'user_tower'

    base_path: '/models/user_tower'

    model_platform: 'tensorflow'

    model_version_policy {

      specific { versions: 1 versions: 2 } # Keep 2 versions for rollback

    }

  }

  config {

    name: 'ranking_model'
```

```
    base_path: '/models/ranking'

    model_platform: 'tensorflow'

    model_version_policy { latest { num_versions: 2 } }

  }
}
```

```
# batching_config.pbtxt

max_batch_size { value: 128 }

batch_timeout_micros { value: 5000 } # 5ms timeout

num_batch_threads { value: 8 }

max_enqueued_batches { value: 100 }
```

Docker Deployment

```
# Dockerfile.serving

FROM tensorflow/serving:2.15.0-gpu


COPY models /models

COPY config/model_config.pbtxt /config/model_config.pbtxt

COPY config/batching_config.pbtxt /config/batching_config.pbtxt


ENV MODEL_NAME=recommendation

ENV TF_CPP_MIN_LOG_LEVEL=1
```

```
ENTRYPOINT ["tensorflow_model_server"]

CMD ["--port=8500", "--rest_api_port=8501", \

    "--model_config_file=/config/model_config.pbtxt", \

    "--enable_batching=true", \

    "--batching_parameters_file=/config/batching_config.pbtxt"]
```

13.2.2 NVIDIA Triton Inference Server

Triton provides multi-framework support (TensorFlow, PyTorch, ONNX), advanced batching, and GPU optimization. It is ideal for heterogeneous model deployments.

Triton Model Configuration

```
# config.pbtxt for ranking model

name: "ranking_model"

platform: "pytorch_libtorch"

max_batch_size: 256

input [

    { name: "user_features" dims: [64] data_type: TYPE_FP32 },

    { name: "item_features" dims: [64] data_type: TYPE_FP32 },

    { name: "cross_features" dims: [32] data_type: TYPE_FP32 }

]

output [

    { name: "score" dims: [1] data_type: TYPE_FP32 }

]
```

```
instance_group [
    { count: 2 kind: KIND_GPU gpus: [0, 1] }
]

dynamic_batching {
    preferred_batch_size: [32, 64, 128]
    max_queue_delay_microseconds: 5000
}
```

13.3 Embedding Serving with Redis

Pre-computed item embeddings are stored in Redis for sub-millisecond retrieval. This is critical for the candidate generation stage.

Redis Embedding Store

```
import redis

import numpy as np

from typing import List, Optional

class EmbeddingStore:

    def __init__(self, host='redis-cluster', port=6379, db=0):

        self.client = redis.Redis(host=host, port=port, db=db,
decode_responses=False)

        self.embedding_dim = 128
```

```

def set_embedding(self, item_id: str, embedding: np.ndarray, ttl: int =
86400):

    '''Store embedding with 24h TTL.'''

    key = f'emb:{item_id}'

    value = embedding.astype(np.float32).tobytes()

    self.client.setex(key, ttl, value)


def get_embedding(self, item_id: str) -> Optional[np.ndarray]:

    '''Retrieve single embedding.'''

    key = f'emb:{item_id}'

    value = self.client.get(key)

    if value:

        return np.frombuffer(value, dtype=np.float32)

    return None


def get_embeddings_batch(self, item_ids: List[str]) -> dict:

    '''Batch retrieval for efficiency.'''

    keys = [f'emb:{item_id}' for item_id in item_ids]

    values = self.client.mget(keys)

    result = {}

    for item_id, value in zip(item_ids, values):

        if value:

            result[item_id] = np.frombuffer(value, dtype=np.float32)

    return result

```

```
def bulk_load(self, embeddings: dict, batch_size: int = 1000):  
  
    '''Bulk load embeddings using pipeline.'''  
  
    pipe = self.client.pipeline()  
  
    count = 0  
  
    for item_id, embedding in embeddings.items():  
  
        key = f'emb:{item_id}'  
  
        value = embedding.astype(np.float32).tobytes()  
  
        pipe.setex(key, 86400, value)  
  
        count += 1  
  
        if count % batch_size == 0:  
  
            pipe.execute()  
  
            pipe = self.client.pipeline()  
  
    pipe.execute() # Final batch
```


Chapter 14: Real-Time Inference Pipeline

14.1 End-to-End Request Flow

A recommendation request flows through multiple stages: feature assembly, candidate generation, ranking, and post-processing. Each stage must be optimized for latency while maintaining quality.

The target latency budget is typically: Feature retrieval: 10ms, Candidate generation: 30ms, Ranking: 40ms, Post-processing: 10ms, Total: <100ms P99.

14.2 FastAPI Recommendation Service

Complete Recommendation API

```
from fastapi import FastAPI, HTTPException

from pydantic import BaseModel

from typing import List, Optional

import asyncio

import time


app = FastAPI(title='Shorts Recommendation API')


class RecommendationRequest(BaseModel):

    user_id: str

    context: dict = {}

    num_results: int = 20

    exclude_ids: List[str] = []


class RecommendationResponse(BaseModel):
```

```
recommendations: List[dict]

request_id: str

latency_ms: float


class RecommendationService:

    def __init__(self):

        self.feature_store = FeatureStore()

        self.embedding_store = EmbeddingStore()

        self.candidate_index = CandidateIndex()

        self.user_tower = load_user_tower()

        self.ranking_model = load_ranking_model()


    async def get_recommendations(self, request: RecommendationRequest):

        start = time.time()

        # 1. Get user features

        user_features = await
self.feature_store.get_user_features(request.user_id)

        if not user_features:

            user_features = self.get_cold_start_features(request.context)

        # 2. Generate user embedding

        user_embedding = self.user_tower.predict(user_features)
```

```
# 3. Retrieve candidates

scores, candidate_ids = self.candidate_index.search(

    user_embedding, k=1000

)

candidate_ids = [c for c in candidate_ids[0] if c not in
request.exclude_ids][:500]


# 4. Get candidate features

item_features = await
self.feature_store.get_item_features_batch(candidate_ids)


# 5. Rank candidates

ranking_scores = self.rank_candidates(

    user_features, item_features, candidate_ids

)


# 6. Post-process and diversify

final_results = self.post_process(

    candidate_ids, ranking_scores, request.num_results

)


latency = (time.time() - start) * 1000

return final_results, latency


service = RecommendationService()
```

```

@app.post('/recommend', response_model=RecommendationResponse)

async def recommend(request: RecommendationRequest):

    try:

        results, latency = await service.get_recommendations(request)

        return RecommendationResponse(

            recommendations=results,

            request_id=generate_request_id(),

            latency_ms=latency

        )

    except Exception as e:

        raise HTTPException(status_code=500, detail=str(e))

```

14.3 Ranking Pipeline

Efficient Batch Ranking

```

class RankingPipeline:

    def __init__(self, model_client, batch_size=64):

        self.model_client = model_client

        self.batch_size = batch_size

        self.objective_weights = ObjectiveWeights()

    def rank_candidates(self, user_features, item_features_list,
candidate_ids):

        '''Rank candidates with multi-objective scoring.'''

```

```

    all_scores = []

    # Process in batches for efficiency

    for i in range(0, len(candidate_ids), self.batch_size):

        batch_ids = candidate_ids[i:i+self.batch_size]

        batch_features = [item_features_list[cid] for cid in batch_ids]

        # Prepare model input

        model_input = self.prepare_input(user_features, batch_features)

        # Get multi-task predictions

        predictions = self.model_client.predict(model_input)

        # Combine objectives

        batch_scores = self.combine_objectives(predictions)

        all_scores.extend(batch_scores)

    return all_scores

def combine_objectives(self, predictions):
    '''Combine multi-task predictions into final score.'''

    w = self.objective_weights

    scores = (

        w.watch_time * predictions['watch_time'] +

```

```

        w.like_prob * predictions['like_prob'] +

        w.share_prob * predictions['share_prob'] +

        w.skip_prob * predictions['skip_prob']

    )

    return scores.tolist()

```

14.4 Post-Processing & Diversity

Diversity Re-ranking

```

class DiversityReranker:

    def __init__(self, diversity_weight=0.3):

        self.diversity_weight = diversity_weight

    def rerank(self, candidates, scores, item_metadata, k):

        '''MMR-style diversity reranking.'''

        selected = []

        selected_categories = set()

        selected_creators = set()

        remaining = list(zip(candidates, scores))

        while len(selected) < k and remaining:

            best_idx = 0

            best_score = float('-inf')

            for i, (cid, score) in enumerate(remaining):

```

```

        # Relevance component

        relevance = score

        # Diversity penalty

        meta = item_metadata.get(cid, {})

        category = meta.get('category', 'unknown')

        creator = meta.get('creator_id', 'unknown')

        category_penalty = 0.2 if category in selected_categories
else 0

        creator_penalty = 0.3 if creator in selected_creators else 0

        diversity_bonus = 1 - category_penalty - creator_penalty

        # Combined score

        final = (1 - self.diversity_weight) * relevance + \

                self.diversity_weight * diversity_bonus

        if final > best_score:

            best_score = final

            best_idx = i

        # Select best item

        cid, score = remaining.pop(best_idx)

        selected.append({'video_id': cid, 'score': score})

        meta = item_metadata.get(cid, {})

```

```
        selected_categories.add(meta.get('category', 'unknown'))  
  
        selected_creators.add(meta.get('creator_id', 'unknown'))  
  
    return selected
```


Chapter 15: A/B Testing Framework

15.1 Experiment Design Principles

A/B testing is essential for validating recommendation improvements before full deployment. A well-designed experiment framework ensures statistical rigor and minimizes user impact from failed experiments.

Key principles include random assignment based on user ID hash for consistency, sufficient sample size for statistical power, guard rails to automatically stop harmful experiments, and proper metric definition before the experiment starts.

15.2 Experiment Configuration

Experiment System

```
from dataclasses import dataclass

from enum import Enum

from typing import Dict, List, Optional

import hashlib


class ExperimentStatus(Enum):

    DRAFT = 'draft'

    RUNNING = 'running'

    PAUSED = 'paused'

    COMPLETED = 'completed'

    STOPPED = 'stopped' # Stopped due to guard rail


@dataclass

class ExperimentConfig:

    experiment_id: str
```

```

name: str

description: str

status: ExperimentStatus

start_date: str

end_date: str

traffic_percentage: float # 0-100

variants: Dict[str, dict] # variant_name -> config

primary_metric: str

secondary_metrics: List[str]

guard_rails: Dict[str, dict]

```

```
class ExperimentManager:
```

```

    def __init__(self, config_store):

        self.config_store = config_store

        self.experiments = {}

        self.load_active_experiments()

    def load_active_experiments(self):

        configs = self.config_store.get_active_experiments()

        for config in configs:

            self.experiments[config.experiment_id] = config

    def get_variant(self, user_id: str, experiment_id: str) -> Optional[str]:

        '''Deterministic variant assignment based on user ID.'''

```

```

    exp = self.experiments.get(experiment_id)

    if not exp or exp.status != ExperimentStatus.RUNNING:

        return None

    # Hash user_id + experiment_id for consistent assignment

    hash_input = f'{user_id}:{experiment_id}'

    hash_value = int(hashlib.md5(hash_input.encode()).hexdigest(), 16)

    bucket = hash_value % 10000    # 0-9999

    # Check if user is in experiment traffic

    if bucket >= exp.traffic_percentage * 100:

        return None    # Not in experiment

    # Assign to variant

    variant_names = list(exp.variants.keys())

    variant_bucket = bucket % len(variant_names)

    return variant_names[variant_bucket]

def get_user_experiments(self, user_id: str) -> Dict[str, str]:

    '''Get all experiment assignments for a user.'''

    assignments = {}

    for exp_id in self.experiments:

        variant = self.get_variant(user_id, exp_id)

        if variant:

```

```
        assignments[exp_id] = variant

    return assignments
```

15.3 Metrics Collection

Event Logging for Experiments

```
from datetime import datetime

import json


class ExperimentLogger:

    def __init__(self, kafka_producer):

        self.producer = kafka_producer

        self.topic = 'experiment-events'

    def log_impression(self, user_id, video_id, experiment_assignments,
                      position, request_id):

        event = {

            'event_type': 'impression',

            'timestamp': datetime.utcnow().isoformat(),

            'user_id': user_id,

            'video_id': video_id,

            'experiments': experiment_assignments,

            'position': position,

            'request_id': request_id
```

```

    }

    self.producer.send(self.topic, json.dumps(event).encode())

def log_engagement(self, user_id, video_id, experiment_assignments,
                    event_type, watch_time=None):

    event = {

        'event_type': event_type, # 'view', 'like', 'share', 'skip'

        'timestamp': datetime.utcnow().isoformat(),

        'user_id': user_id,

        'video_id': video_id,

        'experiments': experiment_assignments,

        'watch_time_seconds': watch_time

    }

    self.producer.send(self.topic, json.dumps(event).encode())

```

15.4 Statistical Analysis

Experiment Analysis Pipeline

```

import numpy as np

from scipy import stats

class ExperimentAnalyzer:

    def __init__(self, min_samples=1000, confidence_level=0.95):

        self.min_samples = min_samples

```

```

        self.confidence_level = confidence_level

    def analyze_metric(self, control_values, treatment_values):

        '''Perform statistical test on metric.'''

        n_control = len(control_values)

        n_treatment = len(treatment_values)

        if n_control < self.min_samples or n_treatment < self.min_samples:

            return {'status': 'insufficient_data', 'n_control': n_control,

                    'n_treatment': n_treatment}

        mean_control = np.mean(control_values)

        mean_treatment = np.mean(treatment_values)

        relative_lift = (mean_treatment - mean_control) / mean_control * 100

        # Welch's t-test (unequal variances)

        t_stat, p_value = stats.ttest_ind(treatment_values, control_values,

                                           equal_var=False)

        # Confidence interval for difference

        se_diff = np.sqrt(np.var(control_values)/n_control +

                           np.var(treatment_values)/n_treatment)

        z = stats.norm.ppf((1 + self.confidence_level) / 2)

        ci_low = (mean_treatment - mean_control) - z * se_diff

```

```
ci_high = (mean_treatment - mean_control) + z * se_diff
```

```
is_significant = p_value < (1 - self.confidence_level)
```

```
return {
    'status': 'analyzed',
    'mean_control': mean_control,
    'mean_treatment': mean_treatment,
    'relative_lift_pct': relative_lift,
    'p_value': p_value,
    'confidence_interval': [ci_low, ci_high],
    'is_significant': is_significant,
    'n_control': n_control,
    'n_treatment': n_treatment
}
```

```
def check_guard_rails(self, experiment_id, metrics_data, guard_rails):
```

```
    '''Check if experiment should be stopped.'''
```

```
    violations = []
```

```
    for metric_name, config in guard_rails.items():
```

```
        if metric_name not in metrics_data:
```

```
            continue
```

```
        value = metrics_data[metric_name]['relative_lift_pct']
```

```
        if value < config.get('min_lift', -float('inf')):
```

```
        violations.append(f'{metric_name} below minimum: {value:.2f}'
%')

        if value > config.get('max_lift', float('inf')):

            violations.append(f'{metric_name} above maximum: {value:.2f}'
%')

    return violations
```


Chapter 16: Monitoring & Observability

16.1 Monitoring Strategy

Comprehensive monitoring covers four pillars: metrics (quantitative measurements), logs (discrete events), traces (request flow), and alerts (actionable notifications). Each pillar serves different debugging and operational needs.

16.2 Key Metrics to Track

Metric	Description	Alert Threshold	Category
P99 Latency	99th percentile response time	> 150ms	Performance
Error Rate	Percentage of failed requests	> 0.1%	Reliability
QPS	Queries per second	< 80% capacity	Capacity
CTR	Click-through rate on recommendations	< -10% baseline	Quality
Watch Time	Avg watch time per recommendation	< -5% baseline	Quality
Cache Hit Rate	Embedding cache hit percentage	< 90%	Efficiency
Model Staleness	Hours since last model update	> 48 hours	Freshness

16.3 Prometheus Metrics

Metrics Collection Implementation

```
from prometheus_client import Counter, Histogram, Gauge, start_http_server

import time


# Define metrics

REQUEST_COUNT = Counter(

    'recommendation_requests_total',

    'Total recommendation requests',
```

```
        ['endpoint', 'status']
    )

REQUEST_LATENCY = Histogram(
    'recommendation_latency_seconds',
    'Request latency in seconds',
    ['endpoint', 'stage'],
    buckets=[0.01, 0.025, 0.05, 0.1, 0.25, 0.5, 1.0]
)

ACTIVE_REQUESTS = Gauge(
    'recommendation_active_requests',
    'Currently processing requests'
)

MODEL_PREDICTION_COUNT = Counter(
    'model_predictions_total',
    'Total model predictions',
    ['model_name', 'model_version']
)

CACHE_HIT = Counter(
    'embedding_cache_hits_total',
```

```

        'Embedding cache hits',

        ['cache_type']

    )

CACHE_MISS = Counter(

    'embedding_cache_misses_total',

    'Embedding cache misses',

    ['cache_type']

)

class MetricsMiddleware:

    async def __call__(self, request, call_next):

        ACTIVE_REQUESTS.inc()

        start_time = time.time()

        try:

            response = await call_next(request)

            REQUEST_COUNT.labels(

                endpoint=request.url.path,

                status=response.status_code

            ).inc()

            return response

        except Exception as e:

            REQUEST_COUNT.labels(endpoint=request.url.path, status=500).inc()

```

```

        raise

    finally:

        REQUEST_LATENCY.labels(

            endpoint=request.url.path,

            stage='total'

        ).observe(time.time() - start_time)

        ACTIVE_REQUESTS.dec()

```

16.4 Grafana Dashboards

Dashboard Configuration (JSON)

```

{
  "dashboard": {
    "title": "Recommendation System Overview",
    "panels": [
      {
        "title": "Request Rate",
        "type": "graph",
        "targets": [{
          "expr": "rate(recommendation_requests_total[5m])",
          "legendFormat": "{{endpoint}} - {{status}}"
        }]
      },
      {
        "title": "P99 Latency",

```

```

        "type": "graph",

        "targets": [{

            "expr": "histogram_quantile(0.99,
rate(recommendation_latency_seconds_bucket[5m]))",

            "legendFormat": "{{endpoint}}"

        }]

    },

    {

        "title": "Error Rate",

        "type": "stat",

        "targets": [{

            "expr": "sum(rate(recommendation_requests_total{status=~'5..'}
[5m])) / sum(rate(recommendation_requests_total[5m])) * 100"

        }]

    },

    {

        "title": "Cache Hit Rate",

        "type": "gauge",

        "targets": [{

            "expr": "sum(rate(embedding_cache_hits_total[5m])) /
(sum(rate(embedding_cache_hits_total[5m])) +
sum(rate(embedding_cache_misses_total[5m]))) * 100"

        }]

    }

]

}

```

```
}
```

16.5 Alerting Rules

Prometheus Alert Rules

```
groups:

  - name: recommendation-alerts

    rules:

      - alert: HighLatency

        expr: histogram_quantile(0.99,
rate(recommendation_latency_seconds_bucket[5m])) > 0.15

        for: 5m

        labels:

          severity: warning

        annotations:

          summary: 'High P99 latency detected'

          description: 'P99 latency is {{ $value | humanizeDuration }}'

      - alert: HighErrorRate

        expr: sum(rate(recommendation_requests_total{status=~'5..'}[5m])) /
sum(rate(recommendation_requests_total[5m])) > 0.001

        for: 2m

        labels:

          severity: critical

        annotations:

          summary: 'High error rate detected'
```

- alert: LowCacheHitRate

```
    expr: sum(rate(embedding_cache_hits_total[5m])) /  
(sum(rate(embedding_cache_hits_total[5m])) +  
sum(rate(embedding_cache_misses_total[5m]))) < 0.9
```

for: 10m

labels:

severity: warning

annotations:

summary: 'Cache hit rate below 90%'

- alert: ModelStaleness

```
    expr: (time() - model_last_update_timestamp) / 3600 > 48
```

for: 1h

labels:

severity: warning

annotations:

summary: 'Model not updated in 48+ hours'

Chapter 17: MLOps & CI/CD Pipelines

17.1 MLOps Principles

MLOps applies DevOps practices to machine learning systems. Key principles include version control for code, data, and models; automated testing at every stage; continuous training with fresh data; and automated deployment with rollback capabilities.

17.2 Model Registry with MLflow

MLflow Model Registry

```
import mlflow

from mlflow.tracking import MlflowClient


class ModelRegistry:

    def __init__(self, tracking_uri='http://mlflow:5000'):

        mlflow.set_tracking_uri(tracking_uri)

        self.client = MlflowClient()

    def register_model(self, run_id, model_name, metrics, tags=None):

        '''Register trained model with metadata.'''

        model_uri = f'runs:{run_id}/model'

        # Register model version

        mv = mlflow.register_model(model_uri, model_name)

        # Add metadata
```



```

    for key, value in (tags or {}).items():

        self.client.set_model_version_tag(

            model_name, mv.version, key, str(value)

        )

    # Log metrics as description

    desc = ', '.join([f'{k}: {v:.4f}' for k, v in metrics.items()])

    self.client.update_model_version(

        model_name, mv.version, description=desc

    )

    return mv.version


def promote_model(self, model_name, version, stage):

    '''Promote model to stage (Staging/Production).'''

    self.client.transition_model_version_stage(

        model_name, version, stage,

        archive_existing_versions=(stage == 'Production')

    )


def get_production_model(self, model_name):

    '''Load current production model.'''

    return mlflow.pyfunc.load_model(

        model_uri=f'models:{model_name}/Production'
    )

```

)

17.3 Training Pipeline with Kubeflow

Kubeflow Pipeline Definition

```

from kfp import dsl

from kfp.components import create_component_from_func


@dsl.component(base_image='python:3.11', packages_to_install=['polars',
'gcsfs'])

def prepare_training_data(input_path: str, output_path: str, date: str):

    import polars as pl

    df = pl.read_parquet(f'{input_path}/{date}/*.parquet')

    # Data processing logic...

    df.write_parquet(output_path)


@dsl.component(base_image='tensorflow/tensorflow:2.15.0-gpu')

def train_model(data_path: str, model_output: str, params: dict):

    import tensorflow as tf

    # Training logic...

    model.save(model_output)


@dsl.component(base_image='python:3.11')

def evaluate_model(model_path: str, test_data: str) -> dict:

    # Evaluation logic...

```

```
        return {'ndcg': 0.42, 'recall': 0.85}

@dsl.component(base_image='python:3.11')
def register_model(model_path: str, metrics: dict, model_name: str):

    import mlflow

    # Registration logic...

@dsl.pipeline(name='recommendation-training-pipeline')
def training_pipeline(date: str, model_name: str = 'ranking_model'):

    # Step 1: Prepare data

    data_task = prepare_training_data(

        input_path='gs://data-lake/interactions',

        output_path='gs://ml-data/training',

        date=date

    )

    # Step 2: Train model

    train_task = train_model(

        data_path=data_task.output,

        model_output='gs://models/new',

        params={'epochs': 10, 'batch_size': 8192}

    )

    train_task.set_gpu_limit(1)
```

```
# Step 3: Evaluate

eval_task = evaluate_model(

    model_path=train_task.output,

    test_data='gs://ml-data/test'

)


# Step 4: Register if metrics pass

register_model(

    model_path=train_task.output,

    metrics=eval_task.output,

    model_name=model_name

)
```

17.4 GitHub Actions CI/CD

CI/CD Workflow

```
# .github/workflows/ml-pipeline.yml

name: ML Pipeline


on:

  push:

    branches: [main]

    paths: ['src/**', 'config/**']

  schedule:
```

```
- cron: '0 2 * * *' # Daily at 2 AM
```

jobs:

test:

```
runs-on: ubuntu-latest
```

steps:

```
- uses: actions/checkout@v4
```

```
- uses: actions/setup-python@v5
```

with:

```
python-version: '3.11'
```

```
- run: pip install -r requirements-dev.txt
```

```
- run: pytest tests/ -v --cov=src
```

```
- run: python -m mypy src/
```

train:

```
needs: test
```

```
if: github.event_name == 'schedule'
```

```
runs-on: ubuntu-latest
```

steps:

```
- uses: actions/checkout@v4
```

```
- uses: google-github-actions/auth@v2
```

with:

```
credentials_json: ${ secrets.GCP_SA_KEY }
```

```
- run: |
```

```
    pip install kfp

    python scripts/trigger_training.py

deploy:

  needs: test

  if: github.ref == 'refs/heads/main'

  runs-on: ubuntu-latest

  steps:

    - uses: actions/checkout@v4

    - uses: google-github-actions/auth@v2

    with:

      credentials_json: ${ secrets.GCP_SA_KEY }

    - run: |

      gcloud container clusters get-credentials prod-cluster

      kubectl apply -f k8s/

      kubectl rollout status deployment/recommendation-api
```

17.5 Kubernetes Deployment

Kubernetes Manifests

```
# k8s/deployment.yaml

apiVersion: apps/v1

kind: Deployment

metadata:
```

```
name: recommendation-api

spec:

  replicas: 3

  selector:

    matchLabels:

      app: recommendation-api

  template:

    metadata:

      labels:

        app: recommendation-api

    spec:

      containers:

        - name: api

          image: gcr.io/project/recommendation-api:latest

          ports:

            - containerPort: 8080

          resources:

            requests:

              memory: '2Gi'

              cpu: '1000m'

            limits:

              memory: '4Gi'

              cpu: '2000m'

          env:
```

```
- name: REDIS_HOST

  value: redis-cluster

- name: MODEL_SERVER

  value: triton-server:8001

livenessProbe:

  httpGet:

    path: /health

    port: 8080

    initialDelaySeconds: 10

  readinessProbe:

    httpGet:

      path: /ready

      port: 8080

---

apiVersion: v1

kind: Service

metadata:

  name: recommendation-api

spec:

  selector:

    app: recommendation-api

  ports:

    - port: 80

      targetPort: 8080
```

```
apiVersion: autoscaling/v2

kind: HorizontalPodAutoscaler

metadata:
  name: recommendation-api-hpa

spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: recommendation-api

  minReplicas: 3

  maxReplicas: 20

  metrics:
    - type: Resource

      resource:
        name: cpu

        target:
          type: Utilization
          averageUtilization: 70
```

Appendix E: Deployment Checklist

E.1 Pre-Deployment Checklist

Model Validation:

- ☐ Offline metrics meet minimum thresholds (NDCG > 0.35, Recall@100 > 80%)
- ☐ Model size is within serving constraints
- ☐ Inference latency tested (< 50ms P99)
- ☐ Model has been validated on holdout test set

Infrastructure:

- ☐ Model artifacts uploaded to model registry
- ☐ Feature store populated with required features
- ☐ Redis cache warmed with item embeddings
- ☐ FAISS index built and deployed
- ☐ Load testing completed (target QPS achieved)

Monitoring:

- ☐ Prometheus metrics configured
- ☐ Grafana dashboards deployed
- ☐ Alert rules configured
- ☐ On-call rotation established

Rollback Plan:

☐ Previous model version available

☐ Rollback procedure documented

☐ Rollback tested in staging

E.2 Post-Deployment Checklist

☐ Error rate below threshold ($< 0.1\%$)

☐ Latency within SLA (P99 $< 100\text{ms}$)

☐ Business metrics trending positive (CTR, watch time)

☐ No anomalies in model predictions

☐ A/B test metrics collecting properly

Appendix F: Incident Response Playbook

F.1 Severity Levels

Level	Description	Response Time	Escalation
P0	Complete service outage	Immediate	VP Engineering
P1	Major degradation (>50% users affected)	15 minutes	Engineering Manager
P2	Partial degradation (<50% users)	1 hour	Team Lead
P3	Minor issue, workaround available	4 hours	On-call engineer

F.2 Common Issues & Resolution

Issue: High Latency

Symptoms: P99 latency > 150ms

Possible causes: Model server overload, cache miss spike, feature store latency

Resolution: 1) Check model server CPU/memory 2) Verify cache hit rate 3) Scale up replicas if needed 4) Consider fallback to simpler model

Issue: Recommendation Quality Drop

Symptoms: CTR or watch time drops > 10%

Possible causes: Bad model deployment, stale features, data pipeline issues

Resolution: 1) Check recent deployments 2) Verify feature freshness 3) Compare model versions 4) Rollback if necessary

Issue: Model Server Crash

Symptoms: 5xx errors spike, pods restarting

Possible causes: OOM, corrupted model, infrastructure issue

Resolution: 1) Check pod logs 2) Verify model artifacts 3) Increase memory limits 4) Redeploy from known-good state

F.3 Rollback Procedure

Rollback Steps

1. Identify current and target versions

```
kubectl get deployment recommendation-api -o  
jsonpath='{.spec.template.spec.containers[0].image}'
```

2. Rollback to previous version

```
kubectl rollout undo deployment/recommendation-api
```

3. Verify rollback

```
kubectl rollout status deployment/recommendation-api
```

4. If model-specific, update model version in config

```
kubectl set env deployment/recommendation-api MODEL_VERSION=v1.2.3
```

5. Monitor metrics

- Watch error rate in Grafana

- Verify latency returns to normal

- Check business metrics stabilize

End of Volume 3

Volume 4 covers: Advanced Topics, System Design Interviews, and Case Studies